

242. Valid Anagram

Given two strings *s* and *t*, return true if *t* is an anagram of *s*, and false otherwise.

Example 1:

Input: *s* = "anagram", *t* = "nagaram"

Output: true

Example 2:

Input: *s* = "rat", *t* = "car"

Output: false

Constraints:

- $1 \leq s.length, t.length \leq 5 \cdot 10^4$
- *s* and *t* consist of lowercase English letters.

The screenshot displays the LeetCode interface for the '242. Valid Anagram' problem. On the left, the problem description states: 'Given two strings *s* and *t*, return true if *t* is an anagram of *s*, and false otherwise.' It includes two examples: Example 1 with *s* = "anagram", *t* = "nagaram" resulting in true; and Example 2 with *s* = "rat", *t* = "car" resulting in false. Constraints specify that the lengths of *s* and *t* are between 1 and 5 * 10^4, and they consist of lowercase English letters. On the right, a Python solution is shown, defining a class 'Solution' with a method 'isAnagram' that uses a dictionary to count character frequencies. The bottom right shows the test result as 'Accepted' with a runtime of 0 ms.

Dry run

- *s* = "anagram", *t* = "nagaram"
- Count from *s*: a=3, n=1, g=1, r=1, m=1
- Traverse *t* and decrement each count.
- All counts match, so the answer is True.

Why this is better than sorting

The hash map solution is better because it runs in **O(n)** time, while sorting takes **O(n log n)**. For large inputs, counting frequencies is more direct and avoids the extra work needed to reorder characters.

Complexity of the hash map solution

- Time Complexity: **O(n)** because we traverse both strings once.
- Space Complexity: **O(1)** for this problem because only lowercase English letters are allowed, so the dictionary stores at most 26 keys.
- In a more general case with arbitrary characters, space complexity would be **O(k)**, where *k* is the number of unique characters.

Interview-ready explanation

A concise explanation you can say:

I first compare the lengths of the two strings because anagrams must have the same number of characters. Then I use a hash map to count the frequency of each character in the first string. After that, I traverse the second string and decrement those counts. If a character is missing or any count becomes negative, I return False. Otherwise, the strings are anagrams. This approach takes $O(n)$ time and $O(1)$ space for lowercase English letters.